Credence Resource Management
iResolve — Internal Automation & Intelligence Platform

ETL Parallelization — Spec & Research Documentation

Concurrent 3-Vertical Historical Pull (tmobile, attprod, crmprod)

PART A — RESEARCH & FEASIBILITY FINDINGS

A.1 Goal

Run 7-year historical ETL pulls for all 3 verticals **concurrently**, starting 12pm IST, complete by 6pm IST (6-hour window; original target stated as "within 5 hrs" — treated here as the tighter aspirational bound).

Vertical	Account volume	Status
tmobile prod	~1.4 million	Currently connected; baseline 4-5 hrs sequential today
attprod	~12 million (~8.5x tmobile)	Not yet onboarded
crmprod	~15-16 million (~11x tmobile)	Not yet onboarded

Total: ~28-29 million accounts, 7 years of history, one 6-hour window, run concurrently.

Server also runs the production **website** (multiple modules, not just ETL-related) and a separate app, **Qualiflow** — both must remain fully responsive throughout the ETL run.

A.2 Current Baseline (known facts)

- tmobile (1.4M accounts, ~28 quarterly batches over 7 years): **4-5 hrs sequential**, single-threaded, single process, dedicated hardware.
- Per batch: extract from 3 source tables → transform/aggregate → write to a dedicated shadow table.
- Most batches: ~9-10 min. Last 2-3 batches: 1-2 hrs each, dominated by sequential 3-table extraction.
- Transform/aggregate: ~2-4 min per batch for 300-400k accounts (~25-40% of a light batch's time).
- Destination moving from SQL Server to Postgres.
- Current hardware: **2 vCPU, 16 GB RAM** — shared with website and Qualiflow.

A.3 Key Technical Findings

Threading (Python, within one vertical): GIL-bound. I/O-bound work (DB reads/writes, network) overlaps well across threads since the GIL releases during I/O waits. CPU-bound work (row-by-row transform logic) does not — it serializes regardless of thread count. Vectorized pandas/numpy operations partially escape this since the underlying C code can release the GIL.

Multiprocessing (across verticals): Separate processes get separate GILs and can genuinely run CPU-bound work in parallel — **but only if physical cores exist to run them on simultaneously.** Multiprocessing does not create CPU capacity; it only lets you use capacity that already exists.

The core finding: on 2 vCPUs, "3 fully concurrent, fully parallel verticals" is not achievable no matter how the code is structured. 2 vCPUs = 2 units of real parallel CPU capacity. Running 3 vertical processes (each spawning their own inner thread pools) on 2 cores means at most 2 of the 3 verticals' CPU-bound work executes in parallel at any moment; the third queues via OS scheduling. I/O-bound work overlaps more forgivingly since it doesn't need a dedicated core.

RAM is a secondary but real risk: 16GB shared across 3 concurrent multi-million-row pipelines, plus Postgres `shared_buffers`/WAL activity during concurrent bulk writes, plus the website and Qualiflow's own memory needs, is tight enough that swapping is a realistic failure mode — and swapping degrades performance far worse than CPU contention alone.

A.4 Honest Feasibility Assessment

On current hardware (2 vCPU / 16GB, shared with website + Qualiflow): finishing all 3 verticals concurrently by 6pm is unlikely, and this is a capacity problem, not a code problem.

Reasoning:

- tmobile alone takes 4-5 hrs *today*, unoptimized, with the full server to itself (setting aside that website/Qualiflow already share it).
- attprod and crmprod are 8.5x and 11x tmobile's volume respectively. Even after optimizing tmobile down to a 1-2 hr target, if attprod/crmprod's time scales anywhere near proportionally with volume, each could individually need several multiples of tmobile's optimized time — running **alone**, with full hardware.
- Running all 3 **concurrently**, sharing 2 vCPUs three ways for CPU-bound work, slows each vertical down relative to running alone — the opposite of what concurrency is meant to achieve once core count is the bottleneck.
- No real timing data exists yet for attprod or crmprod (not onboarded), so precise numbers aren't provable — but directionally, moving ~29M accounts of 7-year history through extract+transform+load in 6 hours, split three ways, on 2 cores that are also serving a live website, is a very large ask relative to the measured baseline.

A.5 What Comfortable Hardware Looks Like

This is a reasoned estimate based on the workload shape, not a measured number — treat it as a starting point for sizing discussions, not a guarantee.

Rough target: 8 vCPU / 32 GB RAM.

Reasoning:

- **Website + Qualiflow** need dedicated, unshared headroom to stay responsive under load — budget ~2 vCPUs as a floor, more if the website itself runs heavy queries or has its own concurrent modules under load.
- **3-vertical concurrent ETL** needs real parallel CPU for each vertical's transform step — roughly 1+ vCPU per vertical minimum for genuine parallelism, more comfortably since each vertical also runs inner thread pools for table extraction (I/O-bound, less CPU-hungry but still consumes scheduling overhead).
- **RAM**: 16GB is already tight for one 1.4M-account pull's in-memory DataFrames. Three concurrent pipelines up to 16M accounts each, Postgres overhead, and the website/Qualiflow's own memory needs point toward 32GB as a safer floor, not a luxury.

Before committing budget to a specific instance size, the highest-value next step is empirical, not estimated: 1. Run tmobile's optimized pipeline once and monitor real CPU/RAM usage (`htop`, cloud monitoring) during the run. 2. Separately measure website + Qualiflow's normal peak CPU/RAM usage. 3. Use those two real data points to size the shared server accurately, rather than sizing off a volume-ratio estimate alone.

PART B — IMPLEMENTATION SPEC

B.1 Architecture Overview

Three layers of parallelism, each solving a distinct part of the bottleneck:

1. **Cross-vertical layer** — one OS process per vertical (tmobile, attprod, crmprod), not threads, so each gets genuine CPU parallelism up to hardware limits.
2. **Outer batch layer** (within each vertical's process) — dynamic `ThreadPoolExecutor` work queue across that vertical's ~3-month batches, so heavy batches don't stall idle thread capacity.
3. **Inner table layer** (within each batch) — concurrent extraction from the 3 source tables per batch, since sequential 3-table extraction is the dominant cost in the heaviest batches.

```
Process: tmobile          Process: attprod          Process: crmprod
  ThreadPool(outer)           ThreadPool(outer)           ThreadPool(outer)
    Batch 1..28                 Batch 1..N                  Batch 1..N
      ThreadPool(inner)           ThreadPool(inner)           ThreadPool(inner)
        Table A/B/C extract         Table A/B/C extract         Table A/B/C extract
      → merge → transform → COPY into Postgres shadow table (per batch, per vertical)
```

B.2 Implementation Details

Outer queue (per vertical process):

```
from concurrent.futures import ThreadPoolExecutor, as_completed

with ThreadPoolExecutor(max_workers=N) as executor:
    futures = {executor.submit(run_batch, b): b for b in vertical_batches}
    for future in as_completed(futures):
        batch = futures[future]
        try:
            result = future.result()
            log_success(batch, result)
        except Exception as e:
            log_failure(batch, e)
```

Inner table extraction (per batch):

```
def run_batch(batch):
    with ThreadPoolExecutor(max_workers=3) as inner:
        futures = [inner.submit(extract_table, batch, t) for t in [table_a, table_b, table_c]]
        results = [f.result() for f in futures]
    merged = merge_locally(results)
    transformed = transform_and_aggregate(merged)
    write_to_shadow_table_postgres(batch, transformed)  # via COPY, not row-by-row INSERT
```

Postgres writes: use `COPY` (`psycopg2.copy_expert` or equivalent) for bulk loads, not row-by-row INSERT or unoptimized ORM writes — critical at 12-16M account scale, independent of hardware/CPU constraints.

B.3 Safety Requirements

- Each batch writes to its own shadow table — no cross-batch or cross-vertical write contention.
- No shared mutable state across threads or processes; each thread/process owns its own DB connections.
- Failure isolation: one batch or table-extract failure logs and is skipped/retried, not fatal to the whole run.
- Idempotent batch writes so failed batches can be safely re-run.

B.4 Phased Rollout

1. **Phase 1 — Instrumentation:** log per-batch extract/transform/write timings on tmobile; establish real baseline (not estimate) before touching attprod/crmprod.
2. **Phase 2 — Outer queue parallelism:** dynamic work queue for tmobile batches; measure wall-clock and resource usage.
3. **Phase 3 — Inner table parallelism:** concurrent 3-table extraction, targeting the heavy batches specifically.
4. **Phase 4 — Onboard attprod, then crmprod individually** (not concurrently yet), measuring real timing at their actual volumes — this produces the real numbers Part A's estimates are currently missing.
5. **Phase 5 — Concurrent 3-vertical run**, only after Phases 1-4 give real per-vertical numbers and hardware sizing (Part A.5) has been addressed.

B.5 Open Items Before Finalizing

1. Real attprod/crmprod timing (not yet measured — current figures are volume-ratio extrapolations).
2. Hardware decision — whether to upgrade toward the ~8 vCPU / 32GB estimate, or accept a staggered/partial-concurrency fallback on current hardware.
3. Source DB capacity for attprod/crmprod — separate instances vs shared, connection limits under the added concurrency.
4. Postgres sizing — `max_connections`, disk/WAL throughput for 3 concurrent bulk-COPY streams at this volume.
5. Confirm transform step is vectorized (pandas/numpy) vs row-by-row — affects how hard the CPU/core constraint bites in practice.
6. Website + Qualiflow's actual peak resource usage, to size shared headroom accurately rather than by estimate.

Bottom Line

The single-vertical parallelization plan (tmobile → 1-2 hrs) is sound on current hardware, since it only asks for 2-3x concurrency on I/O-heavy work. **Running all 3 verticals — two of them 8-11x tmobile's volume — fully concurrently within a 6-hour window on 2 vCPU/16GB shared with a live website and Qualiflow is not achievable through code restructuring alone.** The realistic path: get real timing data on attprod and crmprod individually first, then decide between hardware upgrade, staggered/partial concurrency, or revisiting the 6pm deadline once real numbers are in.

PART C — DETAILED FAST-ETL APPROACH (Assuming Hardware Is Upgraded to Sizing Estimate)

This section assumes the ~8-10 vCPU / 32-40GB target from Part A.5 is in place, with website + Qualiflow guaranteed 2-3 vCPUs, leaving roughly 6-7 vCPUs available for the 3-vertical concurrent ETL. Even with headroom, "more hardware" only pays off if the pipeline is designed to use it well — the details below are what turn available cores into actual throughput.

C.1 Core allocation strategy

Don't let all 3 vertical processes free-float across all available cores — pin/soft-limit each:

- **tmobile process:** ~1-1.5 vCPU (smallest volume, finishes fastest, needs the least)
- **attprod process:** ~2-2.5 vCPU
- **crmprod process:** ~2.5-3 vCPU (largest volume, needs the most headroom)

This can be done with OS-level CPU affinity (`taskset` on Linux) or container CPU limits (if running in Docker/Kubernetes) so the 3 processes don't contend unpredictably with each other or with the website. Uneven allocation matches uneven data volume — giving crmprod and tmobile equal cores would waste capacity on the smaller vertical.

C.2 Extraction — the biggest lever at scale

- **Avoid OFFSET-based pagination entirely.** At 12-16M rows, `OFFSET n LIMIT m` gets progressively slower as `n` grows, because the DB still scans and discards all skipped rows. Use **keyset pagination** instead — filter on an indexed column (e.g., `WHERE account_id > last_seen_id ORDER BY account_id LIMIT batch_size`), which stays fast regardless of position in the dataset.
- **Confirm indexes exist on the date/partition column used to slice batches** on all 3 source tables for attprod and crmprod — this was validated implicitly for tmobile through its existing batch performance, but must be explicitly checked for the new, larger verticals before assuming batch times will scale predictably.
- **Keep the intra-batch 3-table concurrent extraction** (from Part B) — still valid and even more valuable at scale, since with more cores available you can safely add a 4th inner worker if a table proves to be a bottleneck on its own.
- **Consider smaller batch windows for attprod/crmprod** (monthly instead of quarterly) — this increases batch count but reduces per-batch memory footprint and gives the work queue finer-grained units to distribute across available threads, improving load balancing across the outer ThreadPoolExecutor.

C.3 Transform — squeeze the CPU-bound step

- **Confirm (don't assume) transform code is fully vectorized.** Any `.apply()` with a custom Python function or row-wise loop should be rewritten using native pandas/numpy vector operations — this is often a 10-50x difference in that step alone, independent of threading.
- **Consider Polars instead of pandas for the transform step**, if profiling in Phase 1 shows transform is a meaningful share of batch time at attprod/crmprod's volume. Polars is written in Rust, uses multiple cores automatically for many operations, and handles larger-than-pandas-comfortable datasets more efficiently — this is a natural fit given the extra cores from the hardware upgrade.
- **Process data in chunks within a batch** rather than loading the full batch into one DataFrame if memory becomes tight at 300-400k+ accounts per batch for the larger verticals — trades a small amount of overhead for avoiding memory spikes that could trigger swapping.

C.4 Load — Postgres-specific tuning for concurrent bulk writes

- **Use `COPY` (binary mode where possible)** for all shadow table writes — already established in Part B, but at scale the binary protocol variant (`COPY ... WITH (FORMAT binary)`) is meaningfully faster than text/CSV format for large volumes.
- **Use `UNLOGGED` tables for shadow/staging tables.** Since shadow tables are transient (dropped/replaced each run), skipping WAL logging significantly speeds up writes — this is safe specifically because shadow tables aren't part of your durability guarantee; only the final merged table needs full logging.
- **Drop indexes on shadow tables before COPY, rebuild after.** Index maintenance during bulk insert is a major slowdown; it's almost always faster to load unindexed and build the index once at the end.
- **Tune Postgres session/server parameters for the load window specifically:**
  - `synchronous_commit = off` for the ETL session (safe for shadow tables, not for anything requiring immediate durability)
  - Increase `maintenance_work_mem` for faster index rebuilds after COPY
  - Increase `max_wal_size` / checkpoint settings to avoid checkpoint storms during heavy concurrent COPY from 3 processes
- **Separate connection pool sizing per vertical process**, sized to the inner concurrency each process uses (e.g., if each vertical process runs 3 inner extraction threads + 1 write connection, budget accordingly) — and confirm total across all 3 processes stays within Postgres's `max_connections`.

C.5 Concurrency model at scale (putting it together)

```
Website + Qualiflow: 2-3 vCPU (dedicated, unshared, serves 10-20 concurrent users)

tmobile process (~1-1.5 vCPU):
  Outer ThreadPoolExecutor(3) → batches
    Inner ThreadPoolExecutor(3) → table extraction
      → vectorized transform → COPY (binary, unlogged) → shadow tables

attprod process (~2-2.5 vCPU):
  Outer ThreadPoolExecutor(4-5) → smaller (monthly) batches
    Inner ThreadPoolExecutor(3-4) → table extraction
      → vectorized/Polars transform → COPY (binary, unlogged) → shadow tables

crmprod process (~2.5-3 vCPU):
  Outer ThreadPoolExecutor(4-5) → smaller (monthly) batches
    Inner ThreadPoolExecutor(3-4) → table extraction
      → vectorized/Polars transform → COPY (binary, unlogged) → shadow tables
```

C.6 Optional — asyncio as a further refinement

If, after the above, extraction I/O-wait is still a meaningful share of time (common at this scale, since network/DB round-trip latency doesn't disappear with more cores), consider replacing the inner `ThreadPoolExecutor` table extraction with `asyncio` + an async Postgres driver (`asyncpg`) for the source reads, if the source DB driver supports it. Async I/O can handle many more concurrent in-flight queries per core than OS threads, since it avoids thread context-switch overhead entirely — this becomes worthwhile once you're pushing dozens of concurrent extraction operations across 3 verticals × multiple inner workers each.

C.7 What changes if hardware is *not* upgraded

Everything in C.2 (extraction), C.3 (transform), and C.4 (load) is worth doing **regardless of hardware** — keyset pagination, vectorization, binary COPY, unlogged staging tables, and index deferral are all wins independent of core count. Only C.1 (core allocation across 3 concurrent processes) and C.6 (pushing concurrency further with asyncio) specifically depend on having more than 2 vCPUs to spend. If hardware stays at 2 vCPU/16GB, the software-level optimizations in C.2-C.4 are still worth implementing — they'd just be applied to a staggered or partially-concurrent run rather than a fully concurrent 3-vertical one.